Hybrid-INoT: гибридная агентная архитектура с внутренней ролевой сегрегацией для инженерных задач в условиях длинного общего контекста

Даниил Привезенцев[1*]

[1*]Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Внешняя мультиагентная оркестрация на базе больших языковых моделей многократно передаёт общий контекст отдельным ролям — независимым инстансам (экземплярам) языковой модели или отдельным ИИ-агентам, выполняющим самостоятельные обращения к API, — и тем самым создаёт существенные издержки координации. Под издержками координации здесь и далее понимаются дополнительные токены, расходуемые на повторную передачу общего контекста каждой роли и на координационные сообщения между ролями; обзорные работы по мультиагентным LLM-системам [14, 15] указывают, что при нескольких ролях и длинном контексте такие затраты доминируют в общем токенном бюджете и напрямую определяют денежную стоимость решения одной задачи. В задачах программной инженерии с длинным разделяемым контекстом это приводит к быстрому росту токенного расхода. В работе рассматривается **Hybrid-INoT** — гибридная архитектура, в которой ролевой цикл *planner–worker–critic–self-check* развёрнут внутри одного вызова модели, тогда как вовне вынесены только инструментальные действия, верификация и компрессия контекста. Для этой архитектуры задаются операциональная модель, функция маршрутизации `RouteMode` и условие останова внутреннего ролевого цикла. В работе сопоставляются архитектуры Hybrid-INoT и Classical-MAS и формулируется теоретическое предположение, что при достаточно длинном общем контексте и наличии нескольких внешних ролей Hybrid-INoT может быть экономичнее Classical-MAS по суммарному токенному расходу; это предположение

подлежит эмпирической проверке в рамках предложенной далее программы экспериментов. Кроме того, получена верхняя оценка расхода токенов в худшем случае и выведено условие экономической эффективности малой модели самопроверки. Помимо этого, в работе предлагаются система метрик токенной эффективности, денежной стоимости и поддерживаемости кода, а также программа эмпирической проверки, включающая пилотное сравнение на HumanEval, абляционный анализ, исследование масштабирования по длине контекста, оценку процедуры самопроверки и расчёт TCO на реальных измерениях.

Keywords: Большие языковые модели, агентные системы, интроспекция ролей, Introspection of Thought, Self-Refine, Reflexion, токенная эффективность, экономика инференса, надёжность кода, экспериментальный протокол

1 Введение

Современный агент для программной инженерии должен последовательно понять постановку задачи, извлечь релевантный контекст из репозитория и построить изменение, которое затем можно проверить тестами или статическим анализом. На практике эти функции часто распределяются между несколькими ролями, причём под *ролью* понимается отдельный инстанс (экземпляр) большой языковой модели или отдельный ИИ-агент с собственной инструкцией и отдельным обращением к API: одна роль строит план, другая предлагает решение, третья критикует промежуточный результат, четвёртая запускает внешнюю проверку. Таким образом, каждая роль — это самостоятельный вызов LLM, а не абстрактная функция внутри одного вызова. Такое разбиение удобно концептуально, но становится затратным, когда все роли вынуждены заново читать один и тот же длинный контекст.

В настоящей работе рассматривается именно этот сценарий: задачи, в которых общими для всех ролей оказываются фрагменты кода, документация, сообщения компилятора и журналы выполнения. Нас интересует не только итоговое качество решения, но и цена выбранной схемы оркестрации в токенах, времени ответа и денежной стоимости вызовов. Поэтому далее статья сопоставляет классическую внешнюю мультиагентную организацию с гибридной схемой, в которой ролевая дискуссия частично переносится внутрь одного вызова модели.

1.1 Проблема многократной передачи контекста и оценка её значимости

В агентных системах на основе больших языковых моделей типичной конфигурацией является внешняя мультиагентная оркестрация (Classical-MAS): отдельные процессы или вызовы реализуют роли планировщика, исполнителя, критика и тестировщика [12–14]. В задачах программной инженерии, где несколько ролей читают одни и те же длинные фрагменты кода, логов и документации, такая схема обладает структурным дефектом: *каждая роль получает копию*

общего контекста $\mathcal{C}_0$ *заново*, а между ролями дополнительно курсируют сообщения координации. Для количественной оценки этого эффекта введём следующие обозначения.

Обозначения к формуле ниже.

$\mathcal{A}$ — множество ролей в схеме Classical-MAS, каждая из которых реализована как отдельный вызов большой языковой модели; $|\mathcal{A}|$ — число таких ролей (мощность множества).
$\mathcal{C}_0$ — исходный общий контекст задачи (фрагменты кода, документация, журналы выполнения), одинаковый для всех ролей; $|\mathcal{C}_0|$ — его длина, измеряемая в токенах модели.
$\bar{h}$ — средняя длина одного координационного сообщения между ролями, также в токенах.
D_{ctx} — суммарный токенный расход на повторную передачу общего контекста всем ролям за одну задачу.
H_{coord} — суммарный токенный расход на координационные сообщения, которыми роли обмениваются в процессе решения задачи.
$\Theta(\cdot)$ — стандартная асимптотическая нотация: запись $f = \Theta(g)$ означает, что f растёт не медленнее и не быстрее g с точностью до постоянных множителей.

В терминах введённых обозначений токенный бюджет схемы Classical-MAS содержит слагаемое

$$D_{\text{ctx}} + H_{\text{coord}} \in \Theta\big(|\mathcal{A}| \cdot |\mathcal{C}_0|\big) + \Theta\big(|\mathcal{A}|^2 \cdot \bar{h}\big). \tag{1}$$

Смысл формулы (1) следующий. Первое слагаемое $\Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|)$ описывает *линейный рост* затрат на контекст с числом ролей: если каждая из $|\mathcal{A}|$ ролей перечитывает исходный контекст длины $|\mathcal{C}_0|$, токенный расход на этот этап пропорционален произведению $|\mathcal{A}| \cdot |\mathcal{C}_0|$. Второе слагаемое $\Theta(|\mathcal{A}|^2 \cdot \bar{h})$ описывает *квадратичный рост* координационных затрат: в худшем случае каждая пара ролей может обменяться сообщениями средней длины $\bar{h}$, и число таких пар порядка $|\mathcal{A}|^2$. Таким образом, при увеличении числа ролей затраты на передачу контекста растут линейно, а координационные затраты — квадратично, что делает схему Classical-MAS структурно неэффективной при большом $|\mathcal{A}|$ и длинном $|\mathcal{C}_0|$.

Это не умозрительное наблюдение: эмпирические замеры показывают, что при $|\mathcal{A}| \geq 4$ и $|\mathcal{C}_0| \geq 4000$ токенов доля контекста в общем расходе превышает 60% [14, 15]. При стандартных ценах API \$3/M input-токенов (на уровне Claude Sonnet 3.5 / GPT-4o в 2025 г.) цена этого структурного дефекта составляет единицы тысяч долларов в месяц на скромном объёме в 10 000 задач.

Таким образом, издержки рассматриваемой архитектурной схемы допускают количественное выражение: при масштабе 10^4–10^6 инженерных задач Classical-MAS расходует на многократную ретрансмиссию контекста на порядок больше ресурсов, чем на непосредственную генерацию решения. Именно это обстоятельство определяет актуальность рассматриваемой проблемы.

1.2 Существующие подходы и их ограничения

Открытая литература содержит три родственных линии: (i) внутриконтекстная самокритика одной модели — Self-Refine [7], Reflexion [8]; (ii) чередование рассуждения и действия внутри одного агента — ReAct [6], SWE-agent [17]; (iii) внешняя ролевая декомпозиция на отдельные вызовы — MetaGPT [12], ChatDev [13]. Ни одна из этих работ не задаёт *операционально отличимую* промежуточную конфигурацию, в которой (а) внутри одного вызова удерживается разделяемый контекст без дублирования, (б) ролевая сегрегация достаточна для критического разбора, (в) внешними остаются только операции с проверяемыми сигналами, такими как тесты, статический анализ и вызовы инструментов. Отсутствует также формальное условие, при котором такой гибрид доминирует над чисто внешней схемой Classical-MAS по ожидаемому токенному расходу.

1.3 Основные результаты и вклад статьи

В статье получены следующие результаты:

C1. Операциональное определение Hybrid-INoT с явной сигнатурой состояний, функцией маршрутизации `RouteMode` и условием сходимости внутреннего ролевого цикла (§4).

C2. Утверждение 2 о преимуществе Hybrid-INoT над Classical-MAS по ожидаемому токенному расходу при достаточно длинном общем контексте, доказанное через явный подсчёт токенов при выписанных предположениях (§5).

C3. Неравенство (16), задающее условие экономической эффективности малой модели самопроверки (§5).

C4. Система метрик токенной эффективности, стоимости и поддерживаемости кода с обоснованной формой агрегации (§6).

C5. Программа эмпирической валидации, включающая пять исследований E1–E5 с фиксированными базовыми конфигурациями, абляционным анализом и рассмотрением неблагоприятных сценариев (§7).

Эмпирическая проверка преимуществ Hybrid-INoT при длинном контексте обсуждается в §10.

2 Связанные работы и отличие предлагаемого подхода

2.1 От chain-of-thought к внутреннему ролевому дебату

В работе Wei et al. [5] показано, что явная вербализация промежуточных шагов рассуждения может повышать качество решения на задачах, требующих нескольких связанных логических переходов. В данном контексте под промежуточными шагами понимаются текстовые фрагменты, с помощью которых модель раскладывает сложный вывод на последовательность локальных подзадач, а под качеством — итоговая успешность решения целевой задачи. Подход Self-Refine [7]

формализует цикл «генератор–обратная связь–исправление» внутри одной модели; Reflexion [8] дополняет такую схему механизмом рефлексивной памяти, а ReAct [6] связывает внутреннее рассуждение с обращениями к внешним действиям и инструментам. Работа Sun и Zeng [1] описывает Introspection of Thought (INoT) как структурированный внутренний дебат нескольких ролей. В то же время Turpin et al. [9] показывают, что промежуточный текст цепочки не следует интерпретировать как достоверное описание скрытого вычислительного процесса. Поэтому в настоящей работе интроспекция рассматривается прежде всего как вычислительный механизм отбора кандидатов, критики и селективного перезапуска, а не как буквальное окно в скрытое состояние модели.

2.2 Мультиагентные системы для программной инженерии

Weiss [10] и Wooldridge [11] задают теоретическую рамку MAS. MetaGPT [12] и ChatDev [13] переносят её в контекст больших языковых моделей: роли реализуются как отдельные вызовы, каждая получает собственную копию контекста. Обзорные работы указывают на два ключевых ограничения Classical-MAS: распространение ошибки по цепочке и рост стоимости координации [14, 15]. SWE-bench и SWE-agent [16, 17] показывают, что практическая автоматизация задач программной инженерии требует длинного контекста и итеративного редактирования, что ещё сильнее обостряет проблему ретрансмиссии D_{ctx}.

2.3 Компрессия контекста и внешняя верификация

LongLLMLingua и LLMLingua-2 [20, 21] демонстрируют, что компрессия промпта сохраняет или улучшает качество на длинном контексте при снижении стоимости. Jia et al. [22] применяют этот подход к коду. Для самопроверки кода недостаточно языковой рефлексии: необходимы внешние наблюдаемые сигналы — тесты, статический анализ и сообщения компилятора [23–25].

2.4 Сопоставление Hybrid-INoT с предшественниками

Для систематизации различий между рассматриваемыми архитектурами выделим пять признаков: организацию контекста, структуру ролевого взаимодействия, способ проверки, механизм выбора режима работы и условие остановки.

Таблица 1 показывает, что отдельные компоненты предлагаемой схемы встречаются в предшествующих работах, однако именно их сочетание в рамках единого внутреннего ролевого цикла, дополняемого внешней инструментальной верификацией и явной маршрутизацией, образует архитектуру Hybrid-INoT.

3 Формальная постановка задачи

3.1 Задача

Введём базовые множества, используемые в формальной постановке задачи. Их содержательный смысл — описать, с какими входными и выходными данными работает агент и в каком пространстве задана его случайность.

Таблица 1 Сравнение Hybrid-INoT с близкими архитектурными классами

Признак	Self-Refine / Reflexion	MetaGPT / ChatDev	Hybrid-INoT
Организация контекста	Один общий контекст без явного ролевого разделения.	Для каждой роли формируется отдельная копия общего контекста.	Один общий контекст сохраняется внутри вызова модели и разделяется между внутренними ролями.
Ролевое взаимодействие	Самокритика встроена в цикл одной модели.	Роли разведены по отдельным вызовам и обмениваются сообщениями.	Планирование, генерация и критика выполняются как внутренние роли в одном цикле.
Проверка решений	Преимущественно языковая самооценка.	Преимущественно ролевая языковая проверка; внешняя верификация зависит от реализации.	Итоговый кандидат проходит обязательную внешнюю инструментальную проверку; языковая критика используется как промежуточный фильтр.
Выбор режима работы	Фиксирован архитектурой метода.	Фиксирован архитектурой метода.	Задаётся явной функцией маршрутизации, переключающей внутренний и внешний режимы.
Условие остановки	Фиксированное число итераций или эпизодическое правило останова.	Завершение конвейера ролей или достижение согласия между ролями.	Остановка после успешной внешней проверки либо после достижения предельного числа итераций.

$\mathcal{X}$ — множество *текстовых спецификаций задач*. Каждый элемент $x \in \mathcal{X}$ — это строка естественного языка (например, сформулированная пользователем задача или формулировка задачи в HumanEval).
$\mathcal{C}$ — множество *контекстных артефактов*. Каждый элемент — это конечная последовательность текстовых фрагментов: файлов кода, кусков документации, сообщений компилятора, журналов выполнения. Конкретный контекст задачи $\mathcal{C}_0 \in \mathcal{C}$ — это набор именно таких артефактов, имеющих отношение к задаче.
$\mathcal{Y}$ — множество *кандидатных решений*. Каждый элемент $y \in \mathcal{Y}$ — это текстовый артефакт, например исходный код функции или патч в формате `unified diff`, который затем подаётся во внешнюю проверяемую функцию (запуск тестов, статический анализ). Таким образом, y — не числовой вектор, а именно текст.
$(\Omega, \mathcal{F}, \mathbb{P})$ — *вероятностное пространство*, на котором определены все случайные величины задачи. Здесь Ω — множество возможных исходов одного запуска агента (все возможные сценарии развития процесса решения), $\mathcal{F}$ — σ-алгебра наблюдаемых событий над Ω (формальная структура, позволяющая корректно

говорить о вероятностях событий), $\mathbb{P}$ — вероятностная мера на $\mathcal{F}$, учитывающая сразу три источника случайности: стохастическую генерацию токенов в LLM при температуре > 0, выбор начального числа генератора псевдослучайных чисел (*seed* — целое число, инициализирующее детерминированную последовательность псевдослучайных чисел; фиксация seed обеспечивает воспроизводимость запуска) и недетерминизм внешних инструментальных вызовов (например, таймауты тестов). Проще говоря, $(\Omega, \mathcal{F}, \mathbb{P})$ — это математический способ записать утверждение «запуск агента — это случайный эксперимент».

Определение 1 (Агентная инженерная задача)**.** *Агентная инженерная задача — это кортеж $(x, \mathcal{C}_0, \mathcal{Z}, \mathcal{V})$, где $x \in \mathcal{X}$ — текстовое описание задания; $\mathcal{C}_0 \in \mathcal{C}$ — исходный контекст задачи, включающий код, документацию и журналы выполнения; $\mathcal{Z}$ — конечное множество разрешённых внешних инструментов (запуск тестов, линтер, компилятор, поиск по репозиторию и т. п.); $\mathcal{V}\colon \mathcal{Y} \to \{0, 1\}$ — внешняя проверяемая функция успеха, возвращающая 1 при корректном решении (например, при прохождении всего тестового набора) и 0 иначе.*

Случайные величины, индуцируемые стратегией агента.

Стратегия агента — это зафиксированная процедура, отображающая описание задачи x и контекст $\mathcal{C}_0$ в распределение на $\mathcal{Y}$. Она индуцирует следующие случайные величины (функции из Ω в числовые множества):

$Y\colon \Omega \to \mathcal{Y}$ — *случайный выход агента*: итоговое кандидатное решение, которое агент возвращает в конце работы на конкретном запуске.
$T\colon \Omega \to \mathbb{R}_{\geq 0}$ — *суммарный токенный расход* на одну задачу. Значение $T(\omega)$ — общее число токенов (суммарно по всем вызовам модели во всех ролях и итерациях), израсходованных на решение задачи в конкретном запуске ω. Здесь $\mathbb{R}_{\geq 0}$ — множество неотрицательных действительных чисел.
$C_{inf}\colon \Omega \to \mathbb{R}_{\geq 0}$ — *денежная стоимость инференса* на одну задачу (в валюте расчёта, например в долларах США), вычисляемая как произведение числа входных и выходных токенов на соответствующие тарифы API.
$\ell\colon \Omega \to \mathbb{R}_{\geq 0}$ — *задержка выполнения* одной задачи (wall-clock time в секундах) от получения запроса до возврата решения.

Случайность этих величин обусловлена стохастической генерацией LLM и недетерминизмом внешних инструментов; математическое ожидание $\mathbb{E}[\cdot]$ во всех последующих выражениях берётся по мере $\mathbb{P}$ на $(\Omega, \mathcal{F})$.

Целевая функция и ограничения.

Цель агента — максимизировать ожидаемое качество решения, удерживая ожидаемые затраты в пределах заданных бюджетов. Формально:

$$\max\ \mathbb{E}\big[\mathcal{V}(Y)\big] \quad \text{при условиях} \quad \mathbb{E}[T] \leq B,\ \ \mathbb{E}[C_{\text{inf}}] \leq B_{\$},\ \ \mathbb{E}[\ell] \leq B_{\ell}. \tag{2}$$

Расшифровка всех величин в (2):

$\mathbb{E}[\mathcal{V}(Y)]$ — математическое ожидание функции успеха $\mathcal{V}$ от случайного выхода Y. Поскольку $\mathcal{V} \in \{0, 1\}$, эта величина равна вероятности того, что агент на случайном запуске вернёт корректное решение (то есть $\Pr[\mathcal{V}(Y) = 1]$).
$\mathbb{E}[T]$ — среднее число токенов, расходуемых агентом на одну задачу (по распределению, заданному $\mathbb{P}$).
$\mathbb{E}[C_{inf}]$ — средняя денежная стоимость инференса на одну задачу.
$\mathbb{E}[\ell]$ — средняя задержка ответа на одну задачу.
$B, B_{\$}, B_\ell$ — *верхние бюджетные лимиты*, заданные пользователем или эксплуатационным контекстом: B — допустимый средний токенный расход (в токенах), $B_{\$}$ — допустимая средняя стоимость (в валюте расчёта), B_ℓ — допустимая средняя задержка (в секундах).

Словами: формула (2) означает «среди стратегий, укладывающихся в заданные бюджеты по токенам, деньгам и времени, выбрать ту, у которой наибольшая ожидаемая доля успешных решений».

Сводка обозначений

Ниже собраны все обозначения, используемые в дальнейшем тексте, с явными определениями. Часть из них введена выше при постановке задачи; остальные появятся в разделах об архитектуре и аналитической модели.

$\mathcal{C}_0 \in \mathcal{C}$ — исходный общий контекст задачи (конечная последовательность текстовых артефактов); $|\mathcal{C}_0| \in \mathbb{N}$ — его длина в токенах модели.
$\hat{\mathcal{C}}$ — сжатая версия исходного контекста, используемая во внутреннем цикле Hybrid-INoT; $|\hat{\mathcal{C}}| \leq |\mathcal{C}_0|$.
$\mathcal{A}$ — конечное множество внешних ролей в схеме Classical-MAS, каждая из которых — отдельный вызов LLM; $|\mathcal{A}| \in \mathbb{N}$ — число таких ролей.
$Y \colon \Omega \to \mathcal{Y}$ — случайный выход агента; $y \in \mathcal{Y}$ — конкретный (реализованный) кандидат решения.
$K \in \mathbb{N}$ — число кандидатов, одновременно порождаемых ролью r_{work} на одной итерации внутреннего цикла (базовая конфигурация: $K = 3$).
$T \colon \Omega \to \mathbb{R}_{\geq 0}$ — случайная величина суммарного токенного расхода на одну задачу; $\mathbb{E}[T] = \int_\Omega T(\omega)\, \mathrm{d}\mathbb{P}(\omega)$ — её математическое ожидание, вычисляемое по мере $\mathbb{P}$ на вероятностном пространстве $(\Omega, \mathcal{F}, \mathbb{P})$, введённом выше.
$U_{tok} = Q/T$ — показатель токенной эффективности: отношение агрегированной оценки качества Q (см. §6) к суммарному токенному расходу; единица измерения — «качество на 1000 токенов».
$\tau^\star \in \mathbb{R}_{\geq 0}$ — пороговая длина исходного контекста (в токенах), начиная с которой Hybrid-INoT получает преимущество над Classical-MAS по ожидаемому токенному расходу; её существование утверждается в §5, а эмпирическая оценка — в эксперименте E3 (§7.4).
$\delta_{H1} \in (0, 1)$ — минимальный практически значимый относительный прирост показателя U_{tok}, используемый в гипотезе H1 (базовое значение $\delta_{H1} = 0{,}15$, то есть 15%).

$C^S_{check} \in \mathbb{R}_{\geq 0}$ — денежная стоимость одной предварительной проверки решения, выполняемой *малой* (дешёвой) моделью самопроверки; верхний индекс S обозначает small.
$C^L_{rerun} \in \mathbb{R}_{\geq 0}$ — денежная стоимость одного повторного запуска *крупной* (дорогой) модели; верхний индекс L обозначает large.
$\rho_0, \rho_1 \in [0, 1]$ — вероятности того, что потребуется дорогостоящий повторный запуск крупной модели: ρ_0 — без предварительной проверки малой моделью, ρ_1 — при наличии такой проверки; по построению $\rho_1 \leq \rho_0$.
$\mathbb{E}[\cdot]$ — математическое ожидание на вероятностном пространстве $(\Omega, \mathcal{F}, \mathbb{P})$.
$\mathbb{I}[\,\cdot\,]$ — индикаторная функция: $\mathbb{I}[\mathrm{A}] = 1$, если условие A истинно, и 0 иначе.

3.2 Гипотезы исследования

Перед формулировкой гипотез поясним несколько обозначений и статистических процедур, встречающихся ниже, чтобы избавить читателя от необходимости искать их на стороне.

«п.п.» (процентные пункты) — единица разности двух величин, выраженных в процентах. Если одна конфигурация даёт pass@1 = 68%, а другая = 66%, то разница составляет 2 процентных пункта ($68-66=2$), но при этом относительное изменение равно $\approx 3\%$. В тексте всегда используется «п.п.» именно в этом смысле: абсолютная разность процентов, а не отношение.
Парный критерий Вилкоксона (Wilcoxon signed-rank test) — непараметрический статистический тест для сравнения двух связанных выборок (например, значений метрики на одних и тех же задачах при двух конфигурациях). В отличие от t-критерия не требует нормальности распределения; проверяет гипотезу о симметрии распределения разностей относительно нуля. Применяется стандартно при небольших и негауссовских выборках [28].
Поправка Холма–Бонферрони (Holm–Bonferroni correction) — пошаговая процедура корректировки уровня значимости при множественном сравнении. Если одновременно проверяется m гипотез с p-значениями $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$, то k-я гипотеза отвергается, если $p_{(k)} \leq \alpha/(m-k+1)$. Эта поправка контролирует вероятность хотя бы одной ложной находки (family-wise error rate) при $\alpha = 0{,}05$ и является менее консервативной, чем обычная поправка Бонферрони [29].
bootstrap-доверительный интервал — интервал для оценки параметра, полученный повторным ресэмплированием наблюдаемой выборки с возвращением (10^4 раз) и вычислением нужной статистики на каждом ресэмпле; квантили 2,5% и 97,5% распределения статистики дают 95%-й интервал.
seed (начальное число генератора псевдослучайных чисел) — целое число, которым инициализируется детерминированная последовательность псевдослучайных чисел; фиксация seed при одних и тех же входах гарантирует воспроизводимость запуска LLM и выборки кандидатов.
Synthetic Tool-Use Suite — внутренний синтетический бенчмарк, составленный авторами настоящей работы для проверки гипотезы H3. Он содержит задачи, в которых решение естественно разбивается на несколько *истинно независимых* подзадач, каждая из которых требует вызова внешнего инструмента (запуск

отдельных юнит-тестов, проверка разных файлов линтером, выполнение независимых подзапросов к документации). Точный состав и процедура генерации описаны в Приложении A и в сопроводительном репозитории; в открытых источниках бенчмарк с таким названием отсутствует.

H1 *(преимущество при длинном контексте).* Существует пороговая длина исходного контекста $\tau^\star$ такая, что при $|\mathcal{C}_0| > \tau^\star$ архитектура Hybrid-INoT обеспечивает рост показателя U_{tok} не менее чем на $\delta_{H1} = 15\%$ по сравнению с Classical-MAS, при этом качество решения (pass@1) не ухудшается более чем на 2 п.п. (то есть не более чем на 2 процентных пункта абсолютной разности долей решённых задач). В сравнении *фиксируются*: базовая большая модель (и её идентификатор снимка), набор разрешённых внешних инструментов $\mathcal{Z}$, протокол внешней проверки $\mathcal{V}$, состав и порядок задач, лимит итераций I, число кандидатов K, значения seed, температура генерации и форматы промптов. *Варьируется* только схема оркестрации: в Classical-MAS каждая роль — отдельный вызов LLM, получающий полную копию $\mathcal{C}_0$ и координационные сообщения; в Hybrid-INoT ролевой цикл *planner–worker–critic* развёрнут внутри одного вызова модели над сжатым контекстом $\hat{\mathcal{C}}$, а вовне выносятся только инструменты и верификация.

Проверка: эксперимент E1 (§7.2) с парным критерием Вилкоксона, 95% bootstrap-доверительными интервалами (10^4 ресэмплирований) и поправкой Холма–Бонферрони; значение $\tau^\star$ оценивается в E3 (§7.4).

H2 *(экономическая оправданность малой модели самопроверки).* Существует режим, в котором предварительная проверка, выполняемая малой моделью, уменьшает вероятность дорогостоящего повторного запуска настолько, что выполняется условие (16): $C^S_{\text{check}} < (\rho_0 - \rho_1)\, C^L_{\text{rerun}}$. Содержательный смысл: стоимость одной предварительной проверки, выполняемой дешёвой малой моделью (C^S_{check}), должна быть меньше ожидаемой экономии от снижения частоты дорогих повторных запусков крупной модели. Величина $(\rho_0 - \rho_1)$ — это вероятность того, что повторный запуск был бы нужен без самопроверки, но стал не нужен благодаря ей; умножив её на стоимость одного повторного запуска C^L_{rerun}, получаем ожидаемую экономию на одну задачу. Неравенство выполняется тогда и только тогда, когда вложение в дешёвую проверку окупается.

Проверка: E4 (§7.5) с прямыми измерениями $\rho_0, \rho_1, C^S_{\text{check}}, C^L_{\text{rerun}}$ на 100 задачах HumanEval и 50 задачах SWE-bench Lite.

H3 *(граница применимости).* При задачах с *истинной независимостью* подзадач и *высокой степенью параллелизма* инструментов преимущество Hybrid-INoT по времени ответа исчезает или становится отрицательным. Здесь и далее: *истинная независимость подзадач* — свойство задачи, при котором её можно разбить на несколько частей, каждая из которых решается без обмена промежуточным состоянием и без совместного чтения общего контекста (например, «проверь независимо три разных файла»); формально: существует такое разбиение задачи на n подзадач, что решения любых двух из них не зависят друг от друга и от объединения $\mathcal{C}_0$ за вычетом подмножества, специфичного для конкретной подзадачи. *Высокая степень параллелизма инструментов* — ситуация, когда число одновременно запускаемых внешних инструментов без блокирующих взаимозависимостей составляет не менее двух, то есть коэффициент параллелизма

$p_{\|} = \#\{$одновременно исполняемых инструментальных вызовов$\} \geq 2$. В базовой конфигурации эксперимента E2c используется $p_{\|} \in \{2, 3, 4\}$. При таких условиях Classical-MAS может распределить подзадачи по параллельно работающим ролям и получить линейное ускорение по wall-clock time, тогда как последовательный внутренний цикл Hybrid-INoT этого ускорения не даёт.

Проверка: специальный эксперимент E2c (§7.3) на подмножестве из 50 задач с явно параллельной структурой, взятых из Synthetic Tool-Use Suite.

Порог $\delta_{H1} = 15\%$ для H1 выбран как минимально практически значимое улучшение с учётом типичной вариативности токенного бюджета при изменении seed ($\approx$ 5–8% std в [7]) и инженерных издержек развёртывания гибридной схемы при масштабе 10^4 задач в месяц. Устойчивость выводов к выбору этого порога оценивается в E1 на сетке $\delta \in \{5\%, 10\%, 15\%, 20\%\}$.

4 Метод: Hybrid-INoT

4.1 Определение метода и базовая идея

Прежде чем переходить к формальному описанию, дадим содержательное определение метода, на котором строится предлагаемая архитектура, и сформулируем саму идею Hybrid-INoT в минимально необходимой форме.

Определение 2 (Introspection of Thought, INoT)**.** Introspection of Thought *(INoT) — это приём организации рассуждений большой языковой модели, при котором* внутри одного обращения *к модели разворачивается структурированный диалог нескольких* ролей *(ролевых префиксов: «планировщик», «исполнитель», «критик» и т. п.). Каждая роль читает и дописывает общий рабочий буфер в рамках одного прогона модели, что позволяет удерживать общий контекст без дублирования [1]. В отличие от внешней мультиагентной оркестрации, каждая роль INoT* не *является отдельным вызовом LLM — это ролевой префикс внутри единственного вызова.*

Определение 3 (Hybrid-INoT)**.** Hybrid-INoT *— архитектура, предлагаемая в настоящей работе, которая комбинирует внутренний ролевой цикл* planner–worker–critic *в смысле Определения 2 с* обязательной *внешней инструментальной проверкой и предобработкой контекста. А именно:*

- *роли планирования (r_{plan}), генерации кандидатов (r_{work}) и критики (r_{crit}) выполняются как ролевые префиксы внутри одного вызова крупной LLM над сжатым общим контекстом $\hat{\mathcal{C}}$;*
- *вовне вынесены три и только три класса операций: (i) компрессия контекста $\mathcal{C}_0 \mapsto \hat{\mathcal{C}}$ (отдельным лёгким модулем), (ii) внешняя проверяемая верификация r_{self} (запуск тестов, статического анализа, компилятора), (iii) функция маршрутизации* `RouteMode`*, решающая, какой режим работы применить к данной задаче.*

Таким образом, Hybrid-INoT — это промежуточная конфигурация между чисто внутриконтекстной самокритикой (Self-Refine, Reflexion) и полностью внешней мультиагентной оркестрацией (MetaGPT, ChatDev), у которой внутри *сохраняется разделяемый контекст и ролевая сегрегация, а* снаружи *остаются только операции с проверяемыми, инструментальными сигналами.*

Дальнейшие подпункты формализуют эту идею: §4.2 — состояния и сигнатуры ролей, §4.3 — функция маршрутизации, §4.4 — внутренний цикл и условие его сходимости, §4.5 — компрессия контекста, §4.6 — селективный перезапуск.

4.2 Состояния и сигнатуры ролей

Определим состояние системы $\sigma = (x, \hat{\mathcal{C}}, \mathcal{H}, i, b)$, где $x \in \mathcal{X}$ — текстовое описание задачи (см. Определение 1), $\hat{\mathcal{C}}$ — сжатый общий контекст, $\mathcal{H}$ — история итераций (последовательность ранее рассмотренных планов, кандидатов и их оценок), $i \in \{0, \dots, I\}$ — счётчик итераций, $I \in \mathbb{N}$ — верхний предел числа итераций внутреннего цикла, $b \in \mathbb{N}$ — остаток токенного бюджета в токенах.

Пусть Π — *пространство планов*, то есть множество возможных текстовых планов решения (в базовой реализации — короткий маркированный список шагов на естественном языке). Пусть $K \in \mathbb{N}$ — *число кандидатных решений, одновременно порождаемых ролью r_{work} на одной итерации внутреннего цикла*; в базовой конфигурации $K = 3$. Тогда каждая роль задаётся сигнатурой $r\colon \mathcal{S}_r^{\text{in}} \to \mathcal{S}_r^{\text{out}}$:

$$r_{\text{plan}}\colon (x, \hat{\mathcal{C}}, b) \mapsto \pi \in \Pi, \tag{3}$$

$$r_{\text{work}}\colon (\pi, \hat{\mathcal{C}}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \tag{4}$$

$$r_{\text{crit}}\colon (\mathbf{y}, \hat{\mathcal{C}}) \mapsto (s_1, \dots, s_K) \in [0,1]^K, \tag{5}$$

$$r_{\text{self}}\colon (y \in \mathcal{Y}) \mapsto (v, M) \in \{0,1\} \times [0,1]. \tag{6}$$

Расшифровка сигнатур (3)–(6):

r_{plan} — роль *планировщика*. Входы: текст задачи x, сжатый контекст $\hat{\mathcal{C}}$, остаток бюджета b. Выход: план $\pi \in \Pi$ — последовательность шагов решения.
r_{work} — роль *исполнителя* (генератора кандидатов). Входы: план π, сжатый контекст $\hat{\mathcal{C}}$, число требуемых кандидатов K. Выход: набор $\mathbf{y} = (y_1, \dots, y_K)$ из K кандидатных решений $y_k \in \mathcal{Y}$; запись $\mathcal{Y}^K$ — K-я декартова степень множества $\mathcal{Y}$. Значение K задаётся гиперпараметром архитектуры (в базовой конфигурации $K = 3$) и регулирует разнообразие: при $K = 1$ критик всегда оценивает единственный вариант и роль r_{crit} вырождается; увеличение K повышает вероятность получить хоть одно корректное решение ценой пропорционального роста выходных токенов.
r_{crit} — роль *критика*. Входы: набор кандидатов $\mathbf{y}$ и контекст $\hat{\mathcal{C}}$. Выход: набор скалярных оценок $(s_1, \dots, s_K) \in [0,1]^K$, где s_k — оценка качества k-го кандидата (1 — безусловно принять, 0 — безусловно отвергнуть).

r_{self} — *внешний процесс самопроверки*: запуск тестов, статического анализа и линтера. Вход: один отобранный кандидат y. Выход: пара (v, M), где $v \in \{0, 1\}$ — бинарный вердикт внешней проверки (1 — тесты и проверки пройдены, 0 — нет), а $M \in [0, 1]$ — скалярная оценка поддерживаемости кода (см. формулу (21) в §6).

Роли $r_{\text{plan}}, r_{\text{work}}, r_{\text{crit}}$ реализуются как ролевые префиксы внутри *одного* вызова крупной модели над общим $\hat{\mathcal{C}}$; роль r_{self} — отдельный внешний процесс (запуск тестов, линтера, статического анализатора). Содержательный смысл сигнатур: внутренний цикл вырабатывает план, по нему генерирует K кандидатов, оценивает их критиком и передаёт лучшего на внешнюю инструментальную проверку, которая возвращает единственный объективный сигнал об успехе.

4.3 Функция маршрутизации

Содержательный смысл.

`RouteMode` — это *функция выбора режима работы* Hybrid-INoT для данной задачи. Она отвечает на вопрос: решать задачу внутренним ролевым циклом в рамках одного вызова модели (`internal`) или отдать задачу внешней мультиагентной оркестрации (`external`), поскольку в данной постановке внутренний режим заведомо неэффективен? Решение принимается на основе трёх дешёвых признаков задачи: требуется ли обязательный внешний инструмент, доминирует ли параллелизм подзадач и достаточно ли длинный контекст. Формальное определение — ниже.

Определение 4 (RouteMode)**.** *Пусть $\varphi_{\text{tool}}(x) \in \{0, 1\}$ указывает на обязательность внешнего инструмента, $\varphi_{\text{par}}(x) \in \{0, 1\}$ — на доминирующий параллелизм подзадач, $|\hat{\mathcal{C}}|$ — длина сжатого контекста. Тогда*

$$\texttt{RouteMode}(x, \hat{\mathcal{C}}) = \begin{cases} \texttt{external}, & \varphi_{\text{tool}}(x) = 1 \ \vee \ \varphi_{\text{par}}(x) = 1 \ \vee \ |\hat{\mathcal{C}}| < \tau^{\star}_{\text{route}}, \\ \texttt{internal}, & \text{иначе.} \end{cases}$$

Параметры $\varphi_{\text{tool}}, \varphi_{\text{par}}$ определяются отдельным классификатором над x (в базовой реализации — лёгким классификатором на основе инструкции к модели, в дальнейшем — обученным бинарным классификатором); порог $\tau^{\star}_{\text{route}} = 1024$ токена используется как исходная конфигурация и уточняется в E3.

4.4 Внутренний цикл и условие сходимости

Утверждение 1 (Терминация внутреннего цикла)**.** *Пусть каждая операция внутри одной итерации алгоритма 1 завершается за конечное время, а число итераций ограничено сверху параметром I. Тогда алгоритм завершится не позднее чем после I итераций: он либо вернёт решение сразу после успешной внешней проверки $v_i = 1$, либо остановится раньше по правилу* $\text{stagnation}(s_{i,k^\star} - s_{i-1,k^\star}) < \varepsilon$*, либо после исчерпания лимита итераций вернёт лучший найденный кандидат.*

Algorithm 1 Hybrid-INoT: внутренний интроспективный цикл

Require: задача x, контекст $\mathcal{C}_0$, бюджет B, максимум итераций I, число кандидатов K

Ensure: решение $y^\star$, флаг $v^\star$, оценка $M^\star$

$\mathcal{C} \leftarrow \texttt{PolicySanitize}(x, \mathcal{C}_0)$
$\hat{\mathcal{C}} \leftarrow \texttt{CompressIfNeeded}(\mathcal{C})$ ▷ Алг. 2
$m \leftarrow \texttt{RouteMode}(x, \hat{\mathcal{C}})$ ▷ Опр. 4
if $m = \texttt{external}$ **then return** $\texttt{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$
end if
$s^{\text{best}} \leftarrow -\infty;\ y^{\text{best}} \leftarrow \perp$
for $i = 1$ to I **do**
  $\pi_i \leftarrow r_{\text{plan}}(x, \hat{\mathcal{C}}, B - \text{spent})$
  $\{y_{i,k}\} \leftarrow r_{\text{work}}(\pi_i, \hat{\mathcal{C}}, K)$
  $(s_{i,1}, \ldots, s_{i,K}) \leftarrow r_{\text{crit}}(\{y_{i,k}\}, \hat{\mathcal{C}})$
  $k^\star \leftarrow \arg\max_k s_{i,k};\ y_i \leftarrow y_{i,k^\star}$
  if $s_{i,k^\star} > s^{\text{best}}$ **then** $s^{\text{best}} \leftarrow s_{i,k^\star};\ y^{\text{best}} \leftarrow y_i$
  end if
  $(v_i, M_i) \leftarrow r_{\text{self}}(y_i)$ ▷ внешние тесты и статический анализ
  if $v_i = 1$ **then return** $(y_i, 1, M_i)$
  end if
  $\hat{\mathcal{C}} \leftarrow \texttt{UpdateContextWithFailures}(\hat{\mathcal{C}}, v_i)$
  if $\text{stagnation}(s_{i,k^\star} - s_{i-1,k^\star}) < \varepsilon$ **then break** ▷ см. Утв. 1
  end if
end for
return $(y^{\text{best}}, 0, M(y^{\text{best}}))$

Доказательство Внешний цикл по индексу i содержит не более I проходов. Каждая итерация включает конечный набор шагов: построение плана, генерацию K кандидатов, критическую оценку, внешнюю проверку и обновление контекста. По предположению все эти операции завершаются за конечное время, следовательно, завершается и любая отдельная итерация. Поэтому алгоритм либо прекращается досрочно по инструкции **return** или **break**, либо после I-й итерации достигает последней инструкции **return**. Тем самым время работы внутреннего цикла сверху ограничено конечным числом итераций, а утверждение доказано. □

Замечание

Утверждение 1 гарантирует только завершение алгоритма, но не монотонный рост оценок s_i и не корректность самого критика. Сценарий, в котором критик систематически подтверждает неверные решения, рассматривается отдельно в E2b (§7.3) как одна из угроз валидности.

Algorithm 2 Компрессия длинного контекста

Require: $\mathcal{C}_0$, порог длины τ_{len}
Ensure: $\hat{\mathcal{C}}$

1: **if** $|\mathcal{C}_0| \leq \tau_{\text{len}}$ **then return** $\mathcal{C}_0$
2: **end if**
3: разбить $\mathcal{C}_0$ на смысловые блоки $\{b_j\}$
4: оценить $\text{rel}(b_j)$ с помощью классификатора по типу LLMLingua-2 [21]
5: удалить дубликаты, низкоинформативные логи, вторичную документацию
6: сохранить API-сигнатуры, трассы с ошибками, изменяемые файлы и ограничения задачи
7: **return** $\hat{\mathcal{C}}$ с $|\hat{\mathcal{C}}| \leq \tau_{\text{len}}$

4.5 Компрессия контекста

4.6 Селективный перезапуск

Содержательный смысл.

Селективный перезапуск — механизм экономии токенов и денежного бюджета при неудачной попытке решения. Вместо того чтобы при отказе критика перезапускать всю итерацию с нуля или вообще прекращать работу, процедура отбирает $M \leq K$ лучших по оценке критика кандидатов, поочерёдно прогоняет их через внешнюю верификацию `ExternalVerify` и, если ни один не прошёл, формирует новый промпт *только* на основе *подтверждённых* инструментами сигналов ошибки (сообщений тестов, трасс исключений, вывода статического анализатора), а не субъективных комментариев критика. Это ограничивает распространение языковых галлюцинаций между итерациями и уменьшает число дорогих повторных запусков. Алгоритм 3 формализует процедуру.

Algorithm 3 Селективный перезапуск по подтверждённым сигналам

Require: кандидаты $\{y_1, \ldots, y_K\}$, оценки $\{s_k\}$, бюджет повторов $R \in \mathbb{N}_0$
Ensure: итоговое решение $y^\star$

1: отсортировать $\{y_k\}$ по убыванию s_k; оставить верхние $M \leq K$
2: **for** каждый y_m из отобранных **do**
3: $q_m \leftarrow$ `ExternalVerify`(y_m) ▷ запуск тестов и статического анализа
4: **if** $q_m = 1$ **then return** y_m
5: **end if**
6: **end for**
7: **if** $R > 0$ **then**
8: сформировать новый промпт, включив *только* подтверждённые инструментами сигналы ошибки
9: **return** `SelectiveRerun`$(R-1)$
10: **end if**
11: **return** $\arg\max_k s_k$ ▷ если бюджет перезапусков исчерпан, вернуть лучший по критику

Расшифровка входов и выходов Алгоритма 3.

$\{y_1, \ldots, y_K\} \subset \mathcal{Y}$ — набор кандидатных решений, полученных от роли r_{work} на текущей итерации.
$\{s_1, \ldots, s_K\} \subset [0, 1]$ — соответствующие оценки от роли-критика r_{crit}.
$M \in \{1, \ldots, K\}$ — число кандидатов, допускаемых к *дорогой* внешней проверке (в базовой конфигурации $M = \min(K, 3)$).
$R \in \mathbb{N}_0$ — оставшийся бюджет рекурсивных перезапусков; $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$. При $R = 0$ дальнейшие перезапуски запрещены.
$\texttt{ExternalVerify}(y_m) \in \{0, 1\}$ — внешняя проверка (вызов тестового раннера, линтера, статического анализатора); возвращает 1 только при полном успехе.
$y^\star \in \mathcal{Y}$ — итоговое возвращаемое решение: либо первый прошедший внешнюю проверку кандидат, либо лучший по оценке критика при исчерпании бюджета R.

Почему именно подтверждённые сигналы.

Восстанавливая промпт только из объективных сообщений инструментов, алгоритм обрывает обратную связь «критик подтверждает собственную ошибку → следующее поколение наследует её», которая систематически ухудшает сходимость в чисто языковых Self-Refine/Reflexion-циклах [7, 8].

5 Аналитическая модель токенных и стоимостных затрат

5.1 Структура аналитических соотношений

Анализ в данном разделе включает два типа соотношений. К первому относятся прямые балансовые равенства и неравенства, задающие структуру токенных и денежных затрат. Ко второму относятся утверждения о сравнении архитектур при фиксированных предположениях относительно длины контекста, стоимости координации и внутренней критики. Такое разделение позволяет отличить общие расчётные соотношения от результатов, требующих дополнительных условий.

5.2 Токенные бюджеты Classical-MAS и Hybrid-INoT

Перед формулировкой утверждения о преимуществе по токенам введём обозначения для *всех* слагаемых, из которых складываются бюджеты T_{MAS} и T_{INoT}, и зафиксируем соответствующие предположения. Все величины далее выражены в токенах модели и относятся к одной задаче.

T_{MAS} — суммарный токенный расход агента Classical-MAS на одну задачу (случайная величина по распределению $\mathbb{P}$).
T_{INoT} — суммарный токенный расход агента Hybrid-INoT на одну задачу.
T_{ctx}^{MAS} — вклад повторной передачи общего контекста в бюджет Classical-MAS.
T_{coord}^{MAS} — вклад координационных сообщений между ролями в бюджет Classical-MAS.

T_{gen} — совокупный вклад собственно генерации кандидатов и прочих членов, не зависящих линейно от числа внешних ролей $|\mathcal{A}|$ (одинаково присутствующий в обеих архитектурах и потому сокращающийся при вычитании).
T_{cmp} — стоимость однократной компрессии контекста $\mathcal{C}_0 \mapsto \hat{\mathcal{C}}$.
T_{core}^{add} — дополнительная стоимость внутренней критики в Hybrid-INoT сверх обычной генерации (расход на ролевые префиксы и развёрнутый внутренний дебат).
T_{check} — суммарная стоимость внешних верификаций (запусков тестов/линтера, передачи их вывода обратно в модель) за одну задачу.
T_{rerun} — случайная величина стоимости повторных запусков при провале внешней верификации.
$\alpha \in (0, 1]$ — доля ролей Classical-MAS, читающих полный контекст (см. Предположение 1).
$\gamma \in [0, \alpha|\mathcal{A}|)$ — коэффициент, ограничивающий стоимость компрессии: $T_{\text{cmp}} \leq \gamma|\mathcal{C}_0|$.
$\beta \in [0, \alpha|\mathcal{A}| - \gamma)$ — коэффициент, ограничивающий дополнительную стоимость критики: $T_{\text{core}}^{\text{add}} \leq \beta|\hat{\mathcal{C}}|$.
$I \in \mathbb{N}$ — верхний предел числа итераций в обеих архитектурах.
$\bar{h}$ — средняя длина координационного сообщения между ролями (в токенах).

Неравенства между α, β, γ означают, что стоимость компрессии *и* стоимость внутренней критики *вместе* всё же меньше, чем объём повторно передаваемого контекста всеми ролями Classical-MAS; в противном случае выигрыш структурно невозможен. Эти ограничения подлежат эмпирической проверке при конкретной реализации.

Предположение 1. *Среди ролей $\mathcal{A}$ в схеме Classical-MAS не менее $\alpha|\mathcal{A}|$ ролей, $\alpha \in (0, 1]$, читают общий контекст $\mathcal{C}_0$ целиком на каждой итерации. Это эмпирическое наблюдение для существующих реализаций MetaGPT [12] и ChatDev [13], в которых планировщик, исполнитель и критик получают полную копию контекста для корректной работы.*

Предположение 2. *Длина координационных сообщений ограничена $\bar{h}$ токенами, число итераций в обеих схемах равно I, стоимость компрессии $T_{cmp} \leq \gamma|\mathcal{C}_0|$ с $\gamma < \alpha|\mathcal{A}|$, стоимость внутренней критики $T_{core}^{add} \leq \beta|\hat{\mathcal{C}}|$ с $\beta < \alpha|\mathcal{A}| - \gamma$. Верхние оценки γ, β согласуются с эмпирикой работ по компрессии промпта (LongLLMLingua, LLMLingua-2) [20, 21], где коэффициенты сжатия составляют 3–20×, то есть $\gamma \leq 0{,}3$.*

Утверждение 2 (Токенное преимущество при длинном контексте)**.** *При Предположениях 1–2 и $|\mathcal{A}| \geq 3$ существует порог $\tau^\star > 0$ такой, что при $|\mathcal{C}_0| > \tau^\star$ выполняется*

$$\mathbb{E}[T_{MAS}] - \mathbb{E}[T_{INoT}] \;\geq\; (\alpha|\mathcal{A}| - \gamma - \beta) \cdot |\mathcal{C}_0| + \Theta\big(|\mathcal{A}|^2 \bar{h}\big) - T_{check} - \mathbb{E}[T_{rerun}] \;>\; 0. \quad (7)$$

Расшифровка (7)**.** В левой части — разность *ожидаемых* (то есть усреднённых по случайности запуска) токенных бюджетов двух архитектур. В правой части последовательно: линейное по $|\mathcal{C}_0|$ слагаемое с положительным коэффициентом $\alpha|\mathcal{A}| - \gamma - \beta > 0$ (собственно экономия от однократной передачи сжатого контекста вместо повторной передачи полного); добавочный член $\Theta(|\mathcal{A}|^2\bar{h})$, отражающий сэкономленные координационные сообщения; и два *штрафных* члена T_{check} и $\mathbb{E}[T_{\text{rerun}}]$ — дополнительные расходы Hybrid-INoT на внешнюю верификацию и возможные повторные запуски. Преимущество существует, когда линейный выигрыш по контексту при $|\mathcal{C}_0| > \tau^\star$ перекрывает эти штрафы.

Вывод из предположений Разложим токенный бюджет Classical-MAS на три слагаемых:

$$T_{\text{MAS}} = T_{\text{ctx}}^{\text{MAS}} + T_{\text{coord}}^{\text{MAS}} + T_{\text{gen}}. \tag{8}$$

По Предположению 1 на каждой из I итераций не менее $\alpha|\mathcal{A}|$ ролей перечитывают полный контекст $\mathcal{C}_0$, поэтому

$$T_{\text{ctx}}^{\text{MAS}} \geq I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0|. \tag{9}$$

По Предположению 2 координационные сообщения между парами ролей (число таких пар в худшем случае $\Theta(|\mathcal{A}|^2)$) дают $T_{\text{coord}}^{\text{MAS}} \in \Theta(I \cdot |\mathcal{A}|^2\bar{h})$. Подставляя в (8) и беря математическое ожидание, получаем

$$\mathbb{E}[T_{\text{MAS}}] \geq I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0| + \Theta\big(I \cdot |\mathcal{A}|^2\bar{h}\big) + \mathbb{E}[T_{\text{gen}}]. \tag{10}$$

Для Hybrid-INoT контекст сжимается однократно ($T_{\text{cmp}} \leq \gamma|\mathcal{C}_0|$), внутри одного вызова все роли делят общий $\hat{\mathcal{C}}$ без дублирования, дополнительная стоимость критики ограничена $T_{\text{core}}^{\text{add}} \leq \beta|\hat{\mathcal{C}}| \leq \beta|\mathcal{C}_0|$, а внешняя часть даёт T_{check} и $\mathbb{E}[T_{\text{rerun}}]$. Отсюда

$$\mathbb{E}[T_{\text{INoT}}] \leq I \cdot |\hat{\mathcal{C}}| + T_{\text{cmp}} + T_{\text{core}}^{\text{add}} + T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}] + \mathbb{E}[T_{\text{gen}}]. \tag{11}$$

Заметим, что $I \cdot |\hat{\mathcal{C}}| \leq I \cdot |\mathcal{C}_0|$; в пределе сильной компрессии этот член может оказаться существенно меньше $I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0|$ из (9). Вычитая (11) из (10), общий член $\mathbb{E}[T_{\text{gen}}]$ сокращается, и мы получаем

$$\mathbb{E}[T_{\text{MAS}}] - \mathbb{E}[T_{\text{INoT}}] \geq \big(I\alpha|\mathcal{A}| - I - \gamma - \beta\big) \cdot |\mathcal{C}_0| + \Theta\big(I|\mathcal{A}|^2\bar{h}\big) - T_{\text{check}} - \mathbb{E}[T_{\text{rerun}}]. \tag{12}$$

При фиксированных $I, \alpha, |\mathcal{A}|, \gamma, \beta$ коэффициент при $|\mathcal{C}_0|$ положителен (так как $I\alpha|\mathcal{A}| > \gamma + \beta + I$ при $|\mathcal{A}| \geq 3$ и α, β, γ из Предположения 2). Обозначим этот коэффициент $A > 0$, а отрицательные константы — $B' = T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}]$. Тогда правая часть (12) равна $A \cdot |\mathcal{C}_0| + \Theta(I|\mathcal{A}|^2\bar{h}) - B'$, и она становится положительной при

$$|\mathcal{C}_0| > \frac{B' - \Theta(I|\mathcal{A}|^2\bar{h})}{A} =: \tau^\star,$$

что и даёт существование искомого порога. Поглощая коэффициент I в переопределение α (без потери общности), получаем формулировку (7). □

Область применимости утверждения.

Утверждение 2 задаёт нижнюю оценку разности ожидаемых токенных бюджетов при Предположениях 1–2. При $|\mathcal{A}| = 2$ (например, plan+exec без выделенного критика) знак этой разности может изменяться. Аналогично, при $|\mathcal{C}_0| < \tau^\star$ преимущество Hybrid-INoT не гарантируется. Значение $\tau^\star$ зависит от $\alpha, \beta, \gamma, |\mathcal{A}|, I$ и должно уточняться эмпирически (эксперимент E3, §7.4).

5.3 Верхняя граница токенного расхода

Введём величины, входящие в утверждение о верхней границе:

T_{INoT}^{worst} — верхняя оценка токенного расхода Hybrid-INoT *в худшем случае*, то есть когда все I итераций внутреннего цикла заканчиваются отрицательным вердиктом внешней проверки ($v_i = 0$) и алгоритм 1 доходит до полного исчерпания бюджета итераций.
p^L — число входных (prompt) токенов крупной модели на один вызов (*per one candidate generation*); верхний индекс L обозначает large.
r^L — число выходных (response) токенов крупной модели на один вызов.
T_{rerun}^{unit} — стоимость одного повторного запуска (включая повторную передачу истории и вывода инструментов).

Утверждение 3 (Верхняя граница расхода токенов Hybrid-INoT)**.** *В худшем случае, когда все I итераций заканчиваются $v_i = 0$,*

$$T_{INoT}^{worst} \leq T_{cmp} + I \cdot K \cdot \left(p^L + r^L\right) + I \cdot T_{check} + I \cdot T_{rerun}^{unit}. \tag{13}$$

Вывод оценки Токенный расход каждой из I итераций алгоритма 1 складывается из четырёх независимых составляющих:

1. *Компрессия контекста* — выполняется *один раз* перед началом цикла, вклад ровно T_{cmp} (слагаемое без множителя I).
2. *Генерация K кандидатов ролью r_{work}* — каждая генерация требует $p^L + r^L$ токенов крупной модели (входные плюс выходные), всего $K \cdot (p^L + r^L)$ на итерацию. За I итераций набегает $I \cdot K \cdot (p^L + r^L)$. Слагаемые планирования r_{plan} и критики r_{crit} поглощены в эту же оценку верхней границей (их вклад ограничен в рамках одного вызова, разделяющего контекст).
3. *Внешняя верификация* — на каждой итерации вклад не превосходит T_{check}; за I итераций даёт $I \cdot T_{\text{check}}$.
4. *Повторные запуски* — в худшем случае на каждой итерации возникает один повторный запуск стоимостью $T_{\text{rerun}}^{\text{unit}}$, что за I итераций даёт $I \cdot T_{\text{rerun}}^{\text{unit}}$.

Сложив четыре составляющих, получаем (13). □

Полученная оценка используется как верхнее ограничение при конечном бюджете B: при $T \to B$ алгоритм 1 досрочно возвращает лучшего из найденных кандидатов ($\arg\max_k s_k$), что гарантирует соблюдение бюджета даже в наихудшем сценарии.

5.4 Условие экономической эффективности малой модели самопроверки

Обозначения для формулы стоимости инференса.

Пусть агент в процессе решения одной задачи делает несколько вызовов крупной (large, верхний индекс L) и малой (small, верхний индекс S) моделей с индексом i, пробегающим по вызовам. Для каждого вызова обозначим:

p_i^L, r_i^L — число входных (prompt) и выходных (response) токенов i-го вызова крупной модели.
p_i^S, r_i^S — то же для i-го вызова малой модели.
c_L^{in}, c_L^{out} — цены *крупной* модели за один входной и выходной токен соответственно (в валюте расчёта, обычно долл. США за токен; у провайдеров API обычно указывается цена за 10^6 токенов, и значения пересчитываются делением на 10^6).
c_S^{in}, c_S^{out} — то же для малой модели.

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \tag{14}$$

Формула (14) — прямая балансовая запись: денежная стоимость инференса на одну задачу равна сумме по всем вызовам произведений числа токенов на соответствующий тариф, раздельно для входных/выходных токенов и крупной/малой моделей.

Обозначения для условия эффективности.

Рассмотрим схему, в которой после получения кандидата y крупной моделью он предварительно проверяется *малой* моделью; если малая модель сигнализирует об ошибке, запускается повторная генерация крупной моделью. Введём:

C_{check}^S — денежная стоимость одной предварительной проверки малой моделью (включает входной и выходной токенный расход проверочного вызова, пересчитанный в деньги по (14)).
C_{rerun}^L — денежная стоимость одного повторного запуска крупной модели (при провале текущего кандидата).
ρ_0 — вероятность того, что потребуется дорогостоящий повторный запуск *в базовой схеме без малой модели самопроверки* (то есть доля ошибочных кандидатов, не выловленных ни одним дешёвым механизмом).
ρ_1 — та же вероятность *при наличии* предварительной проверки малой моделью; по построению $0 \le \rho_1 \le \rho_0 \le 1$, причём часть ошибочных кандидатов малая модель отсекает, прежде чем они попадут в крупную.

Вывод условия.

Сравним ожидаемые расходы на одну задачу в двух сценариях.

Сценарий 0 (без малой модели): расход $= 0$ при успехе; с вероятностью ρ_0 возникает повторный запуск крупной модели стоимостью C_{rerun}^L. Ожидаемые дополнительные расходы: $\mathbb{E}_0 = \rho_0 \cdot C_{\text{rerun}}^L$.

Сценарий 1 (с малой моделью): *всегда* платим C_{check}^S за предварительную проверку; дополнительно с вероятностью ρ_1 всё ещё нужен повторный запуск стоимостью C_{rerun}^L. Ожидаемые дополнительные расходы: $\mathbb{E}_1 = C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L$.

Малая модель экономически эффективна тогда и только тогда, когда $\mathbb{E}_1 < \mathbb{E}_0$, то есть

$$C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L \;<\; \rho_0 \cdot C_{\text{rerun}}^L, \tag{15}$$

что после переноса $\rho_1 \cdot C_{\text{rerun}}^L$ в правую часть даёт

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \tag{16}$$

Содержательная расшифровка (16)**.** Левая часть — фиксированная стоимость предварительной проверки на каждой задаче. Правая часть — ожидаемая экономия: разность вероятностей $\rho_0 - \rho_1 \geq 0$ показывает, какую долю дорогих повторов удаётся предотвратить благодаря малой модели; умножив её на стоимость одного повторного запуска C^L_{rerun}, получаем ожидаемую денежную экономию. Неравенство выполняется, когда фиксированный взнос в предварительную проверку меньше ожидаемой экономии — то есть когда вложение в дешёвую проверку окупается. Величины $\rho_0, \rho_1, C^S_{\text{check}}, C^L_{\text{rerun}}$ подлежат прямому эмпирическому измерению в эксперименте E4 (§7.5).

6 Система метрик

6.1 Основные метрики и обоснование формы агрегации

Основными количественными характеристиками работы агента в настоящем исследовании выступают токенная эффективность U_{tok}, денежная эффективность $Q_{\$}$ и доля полностью проверенных решений VCR (Verified Candidate Rate). Приведём сначала определения, затем — расшифровку всех входящих величин.

$$U_{\text{tok}} = \frac{Q}{T}, \tag{17}$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \tag{18}$$

$$\text{VCR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\big[\text{tests}_i = 1 \ \wedge \ \text{static}_i = 1 \ \wedge \ \text{sec}_i = 1\big]. \tag{19}$$

Расшифровка (17)–(19)**.**

Q — агрегированная скалярная оценка качества решения задачи, принимающая значения на $[0, 1]$. По построению Q совпадает с ожидаемым значением функции успеха $\mathbb{E}[\mathcal{V}(Y)]$ (см. (2)), а в случае неоднородного датасета — с взвешенной суммой по классам задач $Q = \sum_b w_b Q_b$, где $Q_b \in [0, 1]$ — средняя успешность по классу b (например, «короткие задачи HumanEval», «длинный контекст SWE-bench»), а $w_b \geq 0$ — доля класса в оцениваемой выборке при $\sum_b w_b = 1$.
T — средний токенный расход на одну задачу (в токенах); $U_{\text{tok}} = Q/T$ — «качество, получаемое с 1000 токенов», то есть показатель токенной эффективности.
C_{inf} — средняя денежная стоимость инференса на одну задачу (см. (14)); $Q_{\$} = Q/C_{\text{inf}}$ — «качество, получаемое на 1 доллар», то есть показатель денежной эффективности.
N — число задач в оцениваемой выборке.
$tests_i \in \{0, 1\}$ — результат прогона автоматического тестового набора по i-й задаче: 1, если *все* тесты пройдены, 0 иначе.
$static_i \in \{0, 1\}$ — результат статического анализа (линтера/типизатора): 1, если не сообщено ни одной ошибки из набора критических правил, 0 иначе.

$sec_i \in \{0, 1\}$ — результат проверки безопасности (например, Bandit / Semgrep для Python, CodeQL для общих целей): 1, если не обнаружено ни одной уязвимости, 0 иначе.
$\mathbb{I}[\,\cdot\,]$ — индикаторная функция: равна 1, если указанное в скобках условие истинно, и 0 иначе. В частности, $\mathbb{I}[\mathrm{static}_i = 1]$ принимает значение 1, если статический анализатор для i-й задачи не выявил критических замечаний, и 0 в противном случае.
$\wedge$ — логическая конъюнкция («и»): выражение $\mathrm{tests}_i = 1 \wedge \mathrm{static}_i = 1 \wedge \mathrm{sec}_i = 1$ истинно лишь тогда, когда *одновременно* выполнены все три условия.

Содержательный смысл VCR. Формула (19) считает *долю* задач, на которых выход агента одновременно прошёл все три класса объективных проверок — функциональные тесты, статический анализ и проверку безопасности. Решение засчитывается только при успехе по всем трём критериям: индикатор в сумме даёт 1, если и только если все три условия истинны, и 0 иначе. Множитель $\frac{1}{N}$ нормирует сумму на число задач N, переводя её в долю от 0 до 1 (в процентах — от 0% до 100%). Таким образом, запись $\frac{1}{N}\sum_i \mathbb{I}[\dots]$ — это обычная *средняя доля успехов*, а не произвольный коэффициент 1/3: тройки условий здесь нет, есть N задач и усреднение по ним.

Почему именно три критерия.

Три компонента индикатора — тесты, статический анализ и проверка безопасности — задают *минимально достаточный* набор объективных сигналов для патча в инженерной постановке. Функциональные тесты фиксируют корректность, статический анализ — отсутствие явных ошибок и нарушений стиля, проверка безопасности — отсутствие уязвимостей. Конъюнкция консервативна: частичный успех (например, «тесты пройдены, но есть уязвимость») не засчитывается, что соответствует практикам промышленного CI/CD.

6.2 Агрегированная метрика успешности и её форма

Для агрегированной оценки успешности используется форма

$$\mathrm{MAS}_i = \min(\mathrm{Success}_i, \mathrm{Verified}_i) \cdot M_i^{\lambda}, \qquad \lambda \in [0, 1], \quad \lambda = 0.5. \tag{20}$$

Обоснование этой конструкции состоит в следующем. Во-первых, величины Success и Verified являются необходимыми условиями засчитывания решения: patch, не прошедший внешнюю проверку, не должен получать высокий итоговый балл. Во-вторых, множитель M_i^{λ} моделирует влияние поддерживаемости на итоговую оценку; параметр λ регулирует силу этого влияния. В базовой конфигурации используется $\lambda = 0.5$, а устойчивость выводов к выбору λ оценивается в E2d.

6.3 Веса поддерживаемости и их обоснование

$$M_i = w_{\mathrm{cc}}(1 - \widehat{\mathrm{CC}}_i) + w_{\mathrm{dup}}(1 - \widehat{\mathrm{Dup}}_i) + w_{\mathrm{warn}}(1 - \widehat{\mathrm{Warn}}_i) + w_{\mathrm{doc}}\widehat{\mathrm{Doc}}_i. \tag{21}$$

Таблица 2 Веса поддерживаемости и их связь с подкатегориями ISO/IEC 25010:2023 [26]

Параметр	Значение	Обоснование
w_{cc}	0.35	ISO 25010 Modifiability; цикломатическая сложность [27] — наиболее прямой прокси стоимости сопровождения.
w_{dup}	0.25	ISO 25010 Reusability; дублирование — ключевой сигнал технического долга.
w_{warn}	0.20	ISO 25010 Analysability; warnings — поверхностный, но надёжный сигнал.
w_{doc}	0.20	ISO 25010 Modifiability; покрытие документацией как прокси понятности.

Анализ чувствительности

В E2d проверяется устойчивость ранжирования конфигураций B0–B3 при вариации $(w_{\mathrm{cc}}, w_{\mathrm{dup}}, w_{\mathrm{warn}}, w_{\mathrm{doc}})$ в $\pm 20\%$ и при $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ в (20). Инверсия ранжирования при таких вариациях рассматривается как признак чувствительности метрики к выбранной параметризации.

6.4 Метрики надёжности

$$\mathrm{SCR} = \frac{\#\{\text{ошибочные кандидаты, выявленные процедурой самопроверки}\}}{\#\{\text{все ошибочные кандидаты}\}}, \tag{22}$$

$$\mathrm{RFR} = \frac{\#\{\text{патчи без отката}\}}{\#\{\text{применённые патчи}\}}. \tag{23}$$

7 Программа эмпирической валидации

7.1 Общие правила

Во всех экспериментах фиксируются идентификаторы задач, значения *seed* (начального числа генератора псевдослучайных чисел, инициализирующего стохастическую генерацию LLM и выбор кандидатов; фиксация seed обеспечивает воспроизводимость запуска при одних и тех же входах), версии моделей с идентификаторами снимков и цены токенов с датой измерения. Для закрытых API без фиксированного идентификатора версии данные выносятся в отдельное приложение с описанием наблюдаемых характеристик. Статистическая обработка включает парный критерий Вилкоксона для непрерывных метрик, критерий Мак-Немара для бинарных исходов, 95% bootstrap-доверительные интервалы (10^4 ресэмплирований) и поправку Холма–Бонферрони.

7.2 E1 — Пилот на HumanEval

Цель: проверить H1 на ограниченной выборке. **Данные:** HumanEval (164 задачи) [18]; для каждой задачи рассматриваются четыре версии с искусственным историческим контекстом длиной $\{512, 2048, 8192\}$ токенов, сгенерированным детерминированным шаблоном; всего $164 \times 4 = 656$ случаев. Значения seed: $\{42, 123, 456, 789, 1337\}$. **Сравниваемые конфигурации:**

- B0 Single-Large (один вызов крупной модели без итераций);
- B1 Single-Large + Self-Refine [7], три итерации;
- B2 Classical-MAS (planner, worker, critic — отдельные вызовы, архитектура в духе MetaGPT);
- B3 Hybrid-INoT ($K = 3$, $I = 3$).

Метрики: pass@1 (доля задач, решённых с первой попытки), T (суммарный токенный расход), U_{tok}, C_{inf} (долл. США на 1000 задач при ценах от {дата}), $Q_{\$}$, время ответа. **Критерий успеха H1:** B3 даёт $\Delta U_{\text{tok}} \geq 15\%$ по сравнению с B2 на подмножестве с $|\mathcal{C}_0| > 2048$, $p < 0.05$ после Holm–Bonferroni; значение pass@1 не ухудшается более чем на 2 п.п. **Критерий неудачи:** при невыполнении этих условий H1 считается неподтверждённой, а выводы формулируются без утверждения о преимуществе Hybrid-INoT.

Таблица 3 задаёт формат представления результатов после выполнения эксперимента.

Таблица 3 Формат представления результатов эксперимента E1

Конфигурация	pass@1	токены/задача	U_{tok}	долл. США/1000
B0	—	—	—	—
B1 Self-Refine	—	—	—	—
B2 Classical-MAS	—	—	—	—
B3 Hybrid-INoT	—	—	—	—

7.3 E2 — Абляционный анализ и неблагоприятные сценарии

E2a. B3 vs B3a (без компрессии), B3b (без критика), B3c (без самопроверки). Данные: HumanEval, $n = 164$. Критерий: каждый компонент должен давать статистически значимый вклад; если $\Delta_{\text{B3}-\text{B3b}}$ незначимо, роль критика исключается из архитектуры. **E2b** (неблагоприятный сценарий — систематическая ошибка критика). Подмножество из 50 задач, где базовая конфигурация B0 ошибается; измеряется, как часто r_{crit} подтверждает неверного кандидата; порог тревоги: $> 20\%$ означает необходимость внешнего обучения критика. **E2c** (неблагоприятный сценарий — параллельные инструменты, H3). 50 задач с явно независимыми подзадачами из Synthetic Tool-Use Suite; сравнивается время ответа B3 и B2; ухудшение времени ответа B3 до 30% считается допустимым. **E2d** (анализ чувствительности метрик). Ранжирование конфигураций при $w \pm 20\%$, $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$.

7.4 E3 — Масштабирование по длине контекста

Цель: эмпирически оценить $\tau^\star$ из Утв. 2. **Данные:** 50 задач HumanEval $\times$ 4 длины контекста $\{256, 1024, 4096, 16384\} = 200$ случаев. **Метод:** построить $U_{\text{tok}}(|\mathcal{C}_0|)$ для B0, B2, B3; найти точку пересечения $U_{\text{tok}}^{B3}(|\mathcal{C}_0|) = U_{\text{tok}}^{B2}(|\mathcal{C}_0|)$. **Критерий:**

$\tau^\star$ должен быть конечен и < 16384; иначе H1 и Утв. 2 не подтверждаются на практически значимом диапазоне.

7.5 E4 — Оценка процедуры самопроверки и H2

Цель: проверить (16). **Данные:** 100 задач HumanEval + 50 SWE-bench Lite [16]. **Измерения:** ρ_0 (доля неверных решений без самопроверки), ρ_1 (с самопроверкой), C^S_{check}, C^L_{rerun}, полученные прямыми измерениями. **Дополнительно:** полнота SCR и соотношение стоимости и полезности процедуры самопроверки. **Критерий H2:** $C^S_{\text{check}} < (\rho_0 - \rho_1)C^L_{\text{rerun}}$ на обоих датасетах с 95% CI.

7.6 E5 — TCO на реальных данных

Цель: заменить синтетический расчёт TCO (старая Таблица 4) наблюдаемыми измерениями. **Данные:** $N = 500$ задач разной длины контекста, B2 и B3. **Измерения:** входные и выходные токены каждой роли, число вызовов API, число перезапусков. C_{inf} определяется по актуальным ценам, для которых фиксируются версия модели и дата измерения. **Экстраполяция:** к 10 000 задач/месяц с CI, *не* с точечной оценкой. **Анализ чувствительности:** изменение цены токена на $\pm 50\%$. **Критерий:** если даже нижняя граница CI $\Delta\text{TCO} < 0$, заявить выгоду нельзя.

8 Экономическая модель

8.1 Формализация TCO

$$\text{TCO}_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \tag{24}$$

Формула (24) задаёт структуру совокупной стоимости владения; её параметризация выполняется на основе измерений, описанных в E5.

8.2 Условия экономической целесообразности

- **Выгоден:** $|\mathcal{C}_0| > \tau^\star$; $|\mathcal{A}| \geq 3$; доля ошибок, выявляемых дешёвой процедурой самопроверки, $\geq 30\%$; отсутствует доминирующий параллелизм.
- **Не выгоден:** $|\mathcal{C}_0| < \tau^\star$; истинная независимость подзадач; требование изоляции привилегий на процесс; модель плохо удерживает ролевой формат (измеряется в E2b).

Количественная оценка этих режимов проводится в экспериментах E1, E3 и E5.

9 Ограничения и угрозы валидности

9.1 Когда внутренняя интроспекция хуже внешней оркестрации

1. **Истинная независимость подзадач.** Параллельный запуск нескольких вызовов большой модели даёт линейное ускорение по времени ответа; внутренний цикл является последовательным.
2. **Необходимость изоляции привилегий.** Выполнение недоверенного кода и работа с секретами требуют процессных границ между ролями, что исключает внутренний режим.
3. **Слабое следование ролевому формату.** Если модель не удерживает ролевые префиксы, критик фактически превращается в генератор с иной инструкцией, и архитектурное преимущество исчезает. Этот риск оценивается в E2b.
4. **Систематическая ошибка критика.** Критик, опирающийся на собственные генерации, может устойчиво одобрять ошибочные кандидаты. Основным механизмом защиты в этом случае остаётся внешняя верификация.
5. **Компрессия, удаляющая критически важный факт.** Алгоритм 2 может исключить редкий, но существенный фрагмент; поэтому в сжатом контексте обязательно сохраняются трассы с ошибками и API-сигнатуры.

9.2 Угрозы валидности

Внутренняя валидность: на результаты могут влиять выбор seed и дрейф API; для уменьшения этого эффекта фиксируются идентификаторы версий моделей и используется набор из пяти значений seed. **Внешняя валидность:** HumanEval содержит короткие синтетические задачи, поэтому перенос выводов на реальные задачи программной инженерии дополнительно проверяется на SWE-bench Lite в E4. **Конструктная валидность:** метрики U_{tok} и $Q_{\$}$ чувствительны к ценам токенов; этот фактор анализируется в E5 при вариации цен на $\pm 50\%$. **Статистическая валидность:** множественные сравнения учитываются поправкой Холма–Бонферрони, а при небольших n основной упор делается на доверительные интервалы.

9.3 Область применимости выводов

Полученные теоретические выводы относятся к задачам программной инженерии с длинным общим контекстом и не переносятся автоматически на иные классы задач. Промежуточные ролевые тексты рассматриваются как вычислительный инструмент, а не как интерпретируемое объяснение поведения модели [9]. Наконец, конкретные гиперпараметры $K = 3$, $I = 3$ и $\lambda = 0.5$ используются как рабочая конфигурация и могут быть дополнительно оптимизированы в последующих исследованиях.

10 Направления дальнейшего исследования

Дальнейшее развитие работы связано прежде всего с полной эмпирической валидацией предложенной архитектуры и уточнением условий её применимости.

10.1 Ближайшие задачи

R1. Реализовать воспроизводимый прототип конфигураций B0 и B3 со сквозным запуском на HumanEval.

R2. Провести пилотное сравнение на HumanEval в соответствии с протоколом E1 и получить первые оценки pass@1, токенного расхода и стоимости.

R3. Выполнить абляционный анализ E2a–E2d, включая оценку роли компрессии, критика, процедуры самопроверки и чувствительности метрик.

R4. Уточнить разграничение с Self-Refine либо путём усиления теоретического анализа, либо через прямое эмпирическое сравнение по U_{tok} на длинном контексте.

R5. Оценить TCO на реальных измерениях для выборки из $N = 500$ задач с доверительными интервалами и анализом чувствительности к ценам токенов.

R6. Калибровать веса таблицы 2 на основе экспертного опроса или дополнительных нормативных источников, сохранив анализ чувствительности.

10.2 Расширение экспериментальной программы

R7. Выполнить E3 и E4 в полном объёме и получить эмпирическую оценку порога $\tau^\star$ с доверительными интервалами.

R8. Расширить набор базовых конфигураций за счёт Reflexion [8], ReAct [6] и SWE-agent [17].

R9. Перенести оценку с SWE-bench Lite на SWE-bench Full на выборке не менее 300 задач.

R10. Провести экспертную оценку качества кода на подмножестве из 50 задач для валидации метрики M_i относительно профессионального суждения.

R11. Исследовать механизмы распространения ошибки и сравнить влияние ошибочного критика в Hybrid-INoT и Classical-MAS.

10.3 Целевые результаты эмпирической проверки

Центральный тезис о преимуществе внутренней ролевой сегрегации при длинном общем контексте может считаться подтверждённым при одновременном выполнении следующих условий:

1. E1: $\Delta U_{\text{tok}}^{B3-B2} \geq 15\%$ на $|\mathcal{C}_0| > 2048$, $p < 0.05$ после Holm–Bonferroni.
2. E1: pass@1 B3 не хуже B2 в пределах допуска на неухудшение 2 п.п.
3. E2a: каждый из компонентов (компрессия, критик, самопроверка) даёт значимый независимый вклад.
4. E3: $\tau^\star$ конечен и $< 16\,384$.
5. E4: $C_{\text{check}}^{S} < (\rho_0 - \rho_1) C_{\text{rerun}}^{L}$ на обоих датасетах с 95% CI.
6. E5: $\Delta\text{TCO} < 0$ при нижней границе CI, устойчиво к $\pm 50\%$ ценам.
7. R4: доказательство или эмпирическая значимость против Self-Refine.

При неполном выполнении этих условий объём выводов должен соотноситься с фактически полученными результатами.

10.4 Фокус исследования

Дальнейшие исследования следует сосредоточить на формальном и эмпирическом разграничении Hybrid-INoT и внешней мультиагентной оркестрации по токенной эффективности в задачах программной инженерии с длинным контекстом. Система метрик поддерживаемости и TCO-анализ при этом выступают как вспомогательные компоненты общей исследовательской программы.

11 Заключение

Статья формализует Hybrid-INoT как промежуточную архитектуру между внутриконтекстной самокритикой и внешней мультиагентной оркестрацией. Для этой архитектуры заданы состояние системы, функция маршрутизации и внутренний ролевой цикл; получено утверждение о токенном преимуществе при длинном контексте и числе ролей не менее трёх, выведена верхняя оценка расхода токенов в худшем случае и предложено условие экономической эффективности малой модели самопроверки. Кроме того, разработана система метрик для оценки токенной эффективности, стоимости и поддерживаемости кода и сформирована программа эмпирической валидации, необходимая для количественной проверки теоретических выводов. В совокупности эти результаты задают формальную основу для дальнейшего сопоставления Hybrid-INoT с Classical-MAS и родственными подходами на задачах программной инженерии.

References

[1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, “Information Needs in Collocated Software Development Teams,” in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, “Collaboration, Information Seeking and Communication,” *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” *ICLR*, 2023.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *NeurIPS*, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," *NeurIPS*, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *NeurIPS*, 2023.

[10] G. Weiss, *Multiagent Systems*. MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *ICLR*, 2024.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," *ACL*, 2024.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *ICLR*, 2024.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *NeurIPS*, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," *ACL*, 2024.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," *ACL Findings*, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics," arXiv:2505.23953, 2025.

[26] ISO/IEC 25010:2023, "Systems and software engineering — SQuaRE — Product quality model," ISO, 2023.

[27] T. J. McCabe, "A Complexity Measure," *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[28] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[29] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian J. Statistics*, vol. 6, no. 2, pp. 65–70, 1979.